

— SOLUTIONS ENGINEERING

Agentic AI workflows

Using asynchronous, distributed agents on an event backbone

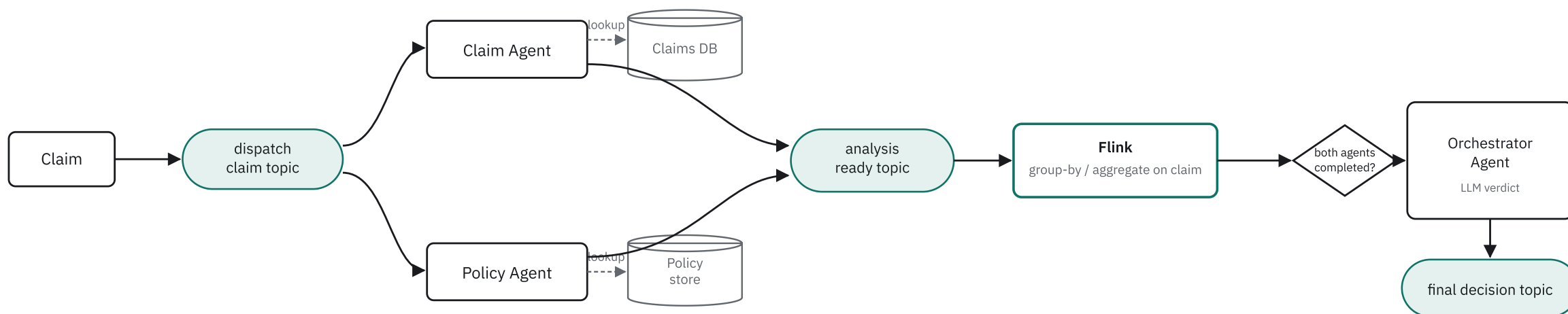
What are agents

- Unlike an LLM on its own, an agent can **take actions**
- An agent is a wrapper around an LLM with **tools, instructions, skills, and a knowledge base**
- Every call to the LLM carries the tools and instructions; the LLM replies with which tool to run. The agent runs it, feeds the result back, and the loop repeats until the task is done
- **Isolation matters.** Too many tools overwhelm the LLM context and cause hallucinations
- Each agent should run in its own **sandbox** to contain security risk

A P&C insurer digitizing claims adjudication

- A **Property & Casualty** carrier settling millions of dollars in claims at very high annual volume
- Adjusters read hundreds of pages per claim and research its merits by hand — **slow and error-prone**
- **Strategic goal:** digitize claims review and approval — cut cost and decision time
- Approach: AI agents that **approve / deny / semi-approve** a claim
- Simplified for this talk — three agents:
 - **Claims Agent** — looks up the claims database for claim details
 - **Policy Agent** — looks up the object store for the policyholder's policy
 - **Coverage Orchestrator** — adjudicates from both agents' outputs
- Each agent runs in its own isolated machine / sandbox — for parallelism and to contain security risk

Flow



The agents never call each other or the orchestrator — they only produce to and consume from topics. Flink does the fan-in; once both agents are in, the orchestrator runs the LLM that reads the claim and the policy to decide.

Hyperscaler agent platforms

Managed agents

FOUNDRY AGENT SERVICE · BEDROCK AGENTCORE

- PRO** Runs on cloud infra with micro-VM sandboxes for isolation
- PRO** Very low code and operational overhead
- CON** Black box and vendor lock-in
- CON** Severe regional capacity limits — forcing customers to run workloads outside their primary business region

Self-hosted agents

AGENT FRAMEWORK, RUN YOURSELF

- PRO** Full control over what the agent does
- PRO** Fine when the use case is simple
- CON** You lose sandbox isolation and the other managed benefits
- CON** The workflow API does not make it run distributed
github.com/rahulbats/workflow-skills-agent-framework-demo

Orchestration frameworks — LangGraph, Temporal

- PRO Strong open-source community
- PRO Workflows are distributed
- CON Built around communication *between* agents — the workers have to know about and report back to the orchestrator / manager

The coordination is point-to-point, and the workflow history lives inside one engine — no other system can subscribe to it.

Can Kafka fill this gap?

- If you already run Kafka for mission-critical event flows, extending it to agent coordination adds **no new system of record**
- Tasks that need parallel or sequential execution are dropped on topics; a distributed swarm of **fully isolated** agents on containers / VMs executes them
- An agent or service crash simply **resumes from the last committed offset**
- Fan-in is a **Flink aggregation / group-by**; the orchestrator picks up the assembled result and runs the final LLM verdict
- No point-to-point communication between agents — the log is the only thing they share, and complex workflows compose without much effort

LIVE

Demo

- Confluent Cloud Kafka + Schema Registry + managed Flink
- Two worker agents, one consumer group each, against a local model(qwen-3.8-26B 4bit quantized)
- Flink barrier: **GROUP BY claim_id**, emit when both agents are in
- Orchestrator consumes the assembled record, runs the LLM, produces the decision

WHAT YOU'LL SEE

```
dispatch CLM-1001 (burst pipe, $8,200)
  → 2 agents run in parallel
  → Flink row: agent_count = 2
  → decision: COVERED
    $8,200 - $1,000 deductible
    = $7,200 payable
```

Future considerations

- The demo is **hand-wired** — an orchestrator and two workers for one use case
- Describe the swarm instead in an **agent-spec.yaml**
- A **Kubernetes operator** watches the spec and launches the swarm, using Kafka as the coordination backbone
- On Kubernetes, **KEDA scales each agent on topic lag** — swarm capacity follows load, including catastrophe-event spikes

 WRAP-UP

Summary & next steps

 THE RECOMMENDATION

- Coordinate the agent swarm on the **event backbone you already run**
- Agents stay yours — your code, your models, isolated
- Flink does the fan-in; the log is the audit trail

 NEXT STEPS

- Scoping workshop on one claim type
- Access to a claims SME and a non-production data feed
- Stand up a shadow-mode pilot — measure decision time and cost